CS-471: Machine Learning

Week 3 — Logistic Regression: Loss Function, Optimisation & Worked Example

Instructor: Dr. Hashir Kiani Source: Lecture 3-1

Comprehensive Study Notes

Week 3

Contents

1 Logistic Regression — Hypothesis Class (Recap & Deep Dive) 2
- 1.1 The Sigmoid Function . . . 2
- 1.2 Properties of the Sigmoid . . . 2
- 1.3 The Derivative of the Sigmoid (Important) . . . 2
- 1.4 Decision Rule . . . 3

2 Logistic Regression — Loss Function 3
- 2.1 Why Not Squared Loss? . . . 3
- 2.2 Binary Cross-Entropy Loss (Log Loss) . . . 3
- 2.3 Understanding the Formula Piece by Piece . . . 3
- 2.4 Training Loss . . . 4
- 2.5 Probabilistic Justification — Maximum Likelihood . . . 4

3 Logistic Regression — Optimisation Algorithm 5
- 3.1 Gradient Descent Update Rule . . . 5
- 3.2 Gradient of the Training Loss . . . 5
- 3.3 Full Update Formula . . . 5
- 3.4 Derivation of the Gradient . . . 6

4 Linear Classification — Worked Example 6
- 4.1 Problem Statement . . . 6
- 4.2 Analysis of the Dataset . . . 7
- 4.3 Hinge Loss GD — Step-by-Step . . . 7
- 4.4 Logistic Regression on the Same Data (Cross-Entropy) . . . 8

5 Connecting Linear Regression, Hinge, and Cross-Entropy 9

6 Practice Questions 9

7 Key Takeaways — Week 3 11

1 Logistic Regression — Hypothesis Class (Recap & Deep Dive)

1.1 The Sigmoid Function

Logistic regression uses the following hypothesis function:

$$\boxed{h(\mathbf{x}) = g\big(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})\big) \qquad \text{where} \qquad g(z) = \frac{1}{1+e^{-z}}}$$

The function $g(z)$ is called the **sigmoid** or **logistic function**.

1.2 Properties of the Sigmoid

Property	Value / Behaviour
Domain	$z \in (-\infty, +\infty)$
Range	$g(z) \in (0, 1)$
$g(0)$	0.5 (maximum uncertainty)
$\lim_{z\to+\infty} g(z)$	1
$\lim_{z\to-\infty} g(z)$	0
Monotone	Strictly increasing
Differentiable	Yes, everywhere
Derivative	$g'(z) = g(z)\,[1 - g(z)]$

1.3 The Derivative of the Sigmoid (Important)

Derivation

$$g'(z) = \frac{d}{dz}\frac{1}{1+e^{-z}}$$

Let $u = 1 + e^{-z}$, so $g = 1/u$.

$$g'(z) = -\frac{1}{u^2}\cdot(-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2}$$

Factoring:

$$g'(z) = \frac{1}{1+e^{-z}}\cdot\frac{e^{-z}}{1+e^{-z}} = g(z)\cdot\frac{1+e^{-z}-1}{1+e^{-z}} = g(z)\cdot(1-g(z))$$

$$\boxed{g'(z) = g(z)\big(1-g(z)\big)}$$

This compact form is used when deriving the gradient of the logistic loss.

1.4 Decision Rule

- $h(\mathbf{x}) = g(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}))$ is interpreted as $P(y = 1 \mid \mathbf{x}; \mathbf{w})$.
- Predict $y = 1$ if $h(\mathbf{x}) > 0.5$, i.e. if $\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) > 0$.
- Predict $y = 0$ (or -1) if $h(\mathbf{x}) \leq 0.5$, i.e. if $\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) \leq 0$.

Key Idea

Why labels 0/1 for Logistic Regression?
When using the sigmoid and the cross-entropy loss below, it is standard to encode the negative class as $y = 0$ (instead of -1 used with hinge loss). This convention makes the probability interpretation and loss formula clean.

Real-Life Analogy: A weather forecaster does not only say "it will rain" or "it will not rain" — they say "there is a 73% chance of rain." Logistic regression is that forecaster. Instead of a hard ± 1 answer, it outputs a probability, giving a more informative prediction.

2 Logistic Regression — Loss Function

2.1 Why Not Squared Loss?

For regression we used squared loss $\frac{1}{2}(h(x) - y)^2$. For logistic regression:

- $h(\mathbf{x})$ is non-linear (sigmoid), so combining it with squared loss creates a *non-convex* objective with many local minima.
- Gradient descent may get trapped — the optimal solution is not guaranteed.

We need a loss that is **convex** when combined with the sigmoid.

2.2 Binary Cross-Entropy Loss (Log Loss)

$$\boxed{\text{Loss}(y,\, h(\mathbf{x})) = -y \log\big(h(\mathbf{x})\big) - (1 - y) \log\big(1 - h(\mathbf{x})\big)}$$

where $y \in \{0, 1\}$ and $h(\mathbf{x}) \in (0, 1)$.

2.3 Understanding the Formula Piece by Piece

The formula has two terms that "switch on" depending on the true label:

y	Active term	Behaviour
1	$-\log(h(\mathbf{x}))$	Loss is 0 when $h \to 1$ (confident correct prediction). Loss $\to \infty$ as $h \to 0$ (very wrong confident prediction).
0	$-\log(1 - h(\mathbf{x}))$	Loss is 0 when $h \to 0$ (confident correct prediction). Loss $\to \infty$ as $h \to 1$ (very wrong confident prediction).

Key Idea

Intuition: Log loss *heavily penalises* confident but wrong predictions. If the true label is $y = 1$ and your model outputs $h = 0.99$ (correctly confident), loss ≈ 0.01 — tiny. But if your model outputs $h = 0.01$ (wrongly confident), loss $= -\log(0.01) \approx 4.6$ — very large. This asymmetry encourages the model to never be very confident in the wrong direction.

Real-Life Analogy: Imagine a student who says "I am 99% sure the answer is B" but the correct answer is A. The teacher penalises this much more than a student who says "I am 60% sure the answer is B" and is wrong. Cross-entropy is that teacher — it rewards calibrated confidence.

2.4 Training Loss

The average log loss over all N training examples:

$$\boxed{\text{TrainLoss}(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y^{(i)}\log\big(h(\mathbf{x}^{(i)})\big) + \big(1 - y^{(i)}\big)\log\big(1 - h(\mathbf{x}^{(i)})\big)\Big]}$$

Variable Definitions:

- N — number of training examples.
- $y^{(i)} \in \{0, 1\}$ — true label of example i.
- $h(\mathbf{x}^{(i)}) = g(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}))$ — model's predicted probability for example i.
- The negative sign makes this a loss to *minimise* (since log of a number in $(0, 1)$ is negative, the negative makes the whole expression positive).

2.5 Probabilistic Justification — Maximum Likelihood

Derivation

Why is cross-entropy the "right" loss? It comes from the principle of **Maximum Likelihood Estimation (MLE)**.
Assume each label is generated as:

$$P(y^{(i)} \mid \mathbf{x}^{(i)}; \mathbf{w}) = h(\mathbf{x}^{(i)})^{y^{(i)}}\ \big[1 - h(\mathbf{x}^{(i)})\big]^{1-y^{(i)}}$$

(When $y = 1$ this gives h; when $y = 0$ this gives $1 - h$.)
The likelihood of the whole training set (assuming i.i.d. samples):

$$\mathcal{L}(\mathbf{w}) = \prod_{i=1}^{N} h(\mathbf{x}^{(i)})^{y^{(i)}}\ \big[1 - h(\mathbf{x}^{(i)})\big]^{1-y^{(i)}}$$

Taking the negative log-likelihood (to turn maximisation into minimisation):

$$-\log \mathcal{L}(\mathbf{w}) = -\sum_{i=1}^{N}\Big[y^{(i)}\log h(\mathbf{x}^{(i)}) + (1 - y^{(i)})\log\big(1 - h(\mathbf{x}^{(i)})\big)\Big]$$

Dividing by N gives exactly the training loss formula above.
Minimising cross-entropy loss = maximising the likelihood of the training data.

3 Logistic Regression — Optimisation Algorithm

3.1 Gradient Descent Update Rule

The standard gradient descent update:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \alpha \, \nabla_{\mathbf{w}} \, \text{TrainLoss}(\mathbf{w})$$

3.2 Gradient of the Training Loss

The gradient of the binary cross-entropy training loss with respect to $\mathbf{w}$ is elegantly simple:

$$\boxed{\nabla_{\mathbf{w}} \, \text{TrainLoss}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \Big[\, h(\mathbf{x}^{(i)}) - y^{(i)} \Big] \, \mathbf{x}^{(i)}}$$

Notation note: The lecture uses $\mathbf{x}^{(i)}$ in the gradient; this is the raw input vector (which may include the bias "1" prepended), equivalent to using the feature vector $\boldsymbol{\phi}(\mathbf{x}^{(i)})$.

3.3 Full Update Formula

$$\boxed{\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \frac{\alpha}{N} \sum_{i=1}^{N} \Big[h(\mathbf{x}^{(i)}) - y^{(i)} \Big] \, \mathbf{x}^{(i)}}$$

Step-by-step reading of the update:

1. For each training example i, compute the **prediction error**: $h(\mathbf{x}^{(i)}) - y^{(i)}$.
   - Positive error $\Rightarrow$ model over-predicted; update will decrease the score for this input.
   - Negative error $\Rightarrow$ model under-predicted; update will increase the score for this input.
2. Multiply the error by the feature vector $\mathbf{x}^{(i)}$ to get the contribution to the gradient.
3. Average over all N examples.
4. Subtract $\alpha\times$ gradient from $\mathbf{w}$.

Key Idea

Structural similarity to linear regression:
Compare with the linear regression gradient: $\frac{1}{N} \sum [\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}) - y^{(i)}] \boldsymbol{\phi}(\mathbf{x}^{(i)})$.
The logistic regression gradient has the *same form*: (prediction − truth) × features, but the prediction is now $h(\mathbf{x}^{(i)}) = g(\mathbf{w}^\top \mathbf{x}^{(i)})$ (passed through sigmoid) rather than

the raw dot product.
This structural similarity makes logistic regression very easy to implement once you have linear regression working.

3.4 Derivation of the Gradient

Derivation

We want $\nabla_{\mathbf{w}}\text{TrainLoss}(\mathbf{w})$. Focus on a single example (drop i superscript).

$$\text{Loss} = -y \log g(z) - (1-y)\log(1-g(z)), \quad z = \mathbf{w}^\top \mathbf{x}$$

By chain rule: $\nabla_{\mathbf{w}}\text{Loss} = \frac{d\,\text{Loss}}{dg} \cdot g'(z) \cdot \mathbf{x}$
Step 1 — dL/dg:

$$\frac{d\,\text{Loss}}{dg} = -\frac{y}{g} + \frac{1-y}{1-g} = \frac{-y(1-g)+(1-y)g}{g(1-g)} = \frac{g-y}{g(1-g)}$$

Step 2 — multiply by $g'(z) = g(1-g)$:

$$\frac{d\,\text{Loss}}{dg} \cdot g'(z) = \frac{g-y}{g(1-g)} \cdot g(1-g) = g - y = h(\mathbf{x}) - y$$

Step 3 — multiply by $\mathbf{x}$ (from chain rule):

$$\nabla_{\mathbf{w}}\text{Loss} = \big(h(\mathbf{x}) - y\big)\mathbf{x}$$

Average over N examples gives the training gradient above. Note how the $g(1-g)$ terms cancel perfectly — this is why sigmoid paired with cross-entropy is so elegant.

4 Linear Classification — Worked Example

4.1 Problem Statement

The lecture provides the following training dataset for binary classification ($y \in \{-1, +1\}$, using the hinge-loss / sign convention):

i	$\mathbf{x}^{(i)}$	$y^{(i)}$
1	$\begin{bmatrix} 0 \\ 2 \end{bmatrix}$	$+1$
2	$\begin{bmatrix} -2 \\ 0 \end{bmatrix}$	$+1$
3	$\begin{bmatrix} 1 \\ -1 \end{bmatrix}$	-1

Feature extractor (with bias): $\boldsymbol{\phi}(\mathbf{x}) = [1,\ x_1,\ x_2]^\top$

So:

$$\boldsymbol{\phi}^{(1)} = [1,\ 0,\ 2]^\top \qquad y^{(1)} = +1$$
$$\boldsymbol{\phi}^{(2)} = [1,\ -2,\ 0]^\top \qquad y^{(2)} = +1$$
$$\boldsymbol{\phi}^{(3)} = [1,\ 1,\ -1]^\top \qquad y^{(3)} = -1$$

4.2 Analysis of the Dataset

- Examples 1 and 2 (both $y = +1$): x_2 is non-negative and x_1 is non-positive.
- Example 3 ($y = -1$): x_1 is positive and x_2 is negative.
- A classifier that fires on high $x_2 - x_1$ would correctly separate the classes.

4.3 Hinge Loss GD — Step-by-Step

Example

Initialisation: $\mathbf{w} = [w_0,\ w_1,\ w_2]^\top = [0,\ 0,\ 0]^\top$, learning rate $\alpha = 1$.

Iteration 1 — Compute scores and margins:

i	Score $= \mathbf{w}^\top \boldsymbol{\phi}^{(i)}$	$y^{(i)}$	Margin $= y^{(i)} \times$score
1	$0(1) + 0(0) + 0(2) = 0$	$+1$	0
2	$0(1) + 0(-2) + 0(0) = 0$	$+1$	0
3	$0(1) + 0(1) + 0(-1) = 0$	-1	0

All margins $= 0 < 1$, so all three examples contribute to the gradient.

Gradient (using hinge loss rule $\nabla = -\frac{1}{N}\sum_{i:\,\text{margin}<1} y^{(i)}\boldsymbol{\phi}^{(i)}$):

$$\nabla_{w_0} = -\frac{1}{3}\big[(+1)(1) + (+1)(1) + (-1)(1)\big] = -\frac{1}{3}(1 + 1 - 1) = -\frac{1}{3}$$
$$\nabla_{w_1} = -\frac{1}{3}\big[(+1)(0) + (+1)(-2) + (-1)(1)\big] = -\frac{1}{3}(0 - 2 - 1) = +1$$
$$\nabla_{w_2} = -\frac{1}{3}\big[(+1)(2) + (+1)(0) + (-1)(-1)\big] = -\frac{1}{3}(2 + 0 + 1) = -1$$

Weight update ($\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla$):

$$w_0 \leftarrow 0 - 1 \times (-1/3) = +0.33$$
$$w_1 \leftarrow 0 - 1 \times (+1) = -1$$
$$w_2 \leftarrow 0 - 1 \times (-1) = +1$$

$\mathbf{w}^{(1)} = [0.33,\ -1,\ 1]^\top$

Iteration 2 — Recompute scores with updated w:

i	$\boldsymbol{\phi}^{(i)}$	Score	$y^{(i)}$	Margin
1	$[1, 0, 2]$	$0.33(1) + (-1)(0) + 1(2) = 2.33$	$+1$	$+2.33 \geq 1$ ✓
2	$[1, -2, 0]$	$0.33(1) + (-1)(-2) + 1(0) = 2.33$	$+1$	$+2.33 \geq 1$ ✓
3	$[1, 1, -1]$	$0.33(1) + (-1)(1) + 1(-1) = -1.67$	-1	$+1.67 \geq 1$ ✓

All margins ≥ 1 after just one update! The hinge loss gradient is $\mathbf{0}$ — training has converged for this dataset.

Decision Boundary: $\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) = 0 \Rightarrow 0.33 + (-1)x_1 + (1)x_2 = 0 \Rightarrow x_2 = x_1 - 0.33$ This is a line slightly below $x_2 = x_1$. Points above this line get classified $+1$; points below get -1. Verify: example 1 has $x_2 = 2 > 0 - 0.33$ ✓; example 3 has $x_2 = -1 < 1 - 0.33 = 0.67$ ✓.

4.4 Logistic Regression on the Same Data (Cross-Entropy)

Example

Converting to $y \in \{0, 1\}$ convention: $y^{(1)} = 1$, $y^{(2)} = 1$, $y^{(3)} = 0$.
Initialise: $\mathbf{w} = [0, 0, 0]^\top$, $\alpha = 0.5$.

Predictions (all scores are 0 initially): $h^{(1)} = h^{(2)} = h^{(3)} = g(0) = 0.5$.

Errors:

$$\begin{aligned} e^{(1)} &= h^{(1)} - y^{(1)} = 0.5 - 1 = -0.5 \\ e^{(2)} &= h^{(2)} - y^{(2)} = 0.5 - 1 = -0.5 \\ e^{(3)} &= h^{(3)} - y^{(3)} = 0.5 - 0 = +0.5 \end{aligned}$$

Gradient $= \frac{1}{3} \sum e^{(i)} \boldsymbol{\phi}^{(i)}$:

$$\begin{aligned} \nabla_{w_0} &= \frac{1}{3}[(-0.5)(1) + (-0.5)(1) + (0.5)(1)] = \frac{-0.5}{3} \approx -0.167 \\ \nabla_{w_1} &= \frac{1}{3}[(-0.5)(0) + (-0.5)(-2) + (0.5)(1)] = \frac{0 + 1 + 0.5}{3} = \frac{1.5}{3} = 0.5 \\ \nabla_{w_2} &= \frac{1}{3}[(-0.5)(2) + (-0.5)(0) + (0.5)(-1)] = \frac{-1 + 0 - 0.5}{3} = \frac{-1.5}{3} = -0.5 \end{aligned}$$

Update ($\alpha = 0.5$):

$$\begin{aligned} w_0 &\leftarrow 0 - 0.5(-0.167) = +0.083 \\ w_1 &\leftarrow 0 - 0.5(0.5) = -0.25 \\ w_2 &\leftarrow 0 - 0.5(-0.5) = +0.25 \end{aligned}$$

$\mathbf{w}^{(1)} = [0.083,\ -0.25,\ 0.25]^\top$
Verify predictions after update: $\text{Score}^{(3)} = 0.083 - 0.25(1) + 0.25(-1) = 0.083 - 0.5 = -0.417$ $h^{(3)} = g(-0.417) \approx 0.397 < 0.5 \Rightarrow$ predicts 0 (✓).
$\text{Score}^{(1)} = 0.083 + 0.25(2) = 0.583$ $h^{(1)} = g(0.583) \approx 0.642 > 0.5 \Rightarrow$ predicts 1 (✓).
The logistic regression is already classifying all three points correctly after one gradient step.

Alternative Method / Trick

Logistic vs. Hinge Loss — When to use which?

	Hinge Loss	Cross-Entropy (Log Loss)
Model	Linear classifier (± 1)	Logistic regression (0/1)
Output	Hard label	Probability
Loss shape	Piecewise linear	Smooth
Gradient	Zero when margin ≥ 1	Always non-zero
Sparse updates	Yes (only wrong/uncertain examples)	No
Best for	SVMs, margin-based	Probabilistic models, NN outputs

5 Connecting Linear Regression, Hinge, and Cross-Entropy

All three models we have studied follow the same framework:

	Hypothesis	Loss	Gradient
Linear Regression	$h = \mathbf{w}^\top \boldsymbol{\phi}$	$\frac{1}{2}(h-y)^2$	$(h-y)\boldsymbol{\phi}$
Linear Classifier (hinge)	$h = \text{sign}(\mathbf{w}^\top \boldsymbol{\phi})$	$\max\{1 - y \cdot \text{score}, 0\}$	$-y\boldsymbol{\phi}$ if margin < 1; else
Logistic Regression	$h = g(\mathbf{w}^\top \boldsymbol{\phi})$	$-y \log h - (1-y)\log(1-h)$	$(h-y)\boldsymbol{\phi}$

Key Idea

Notice that both linear regression and logistic regression share the gradient form $(h-y)\boldsymbol{\phi}$. The only difference is what h computes: a raw dot product vs. a sigmoid-transformed dot product. This unification is the foundation of the **generalised linear model (GLM)** framework and lies at the heart of neural network design.

6 Practice Questions

Practice Questions

Topic: Sigmoid and Hypothesis

Q1 (Easy): Compute $g(2)$ and interpret the result as a classification decision.

$$g(2) = \frac{1}{1+e^{-2}} = \frac{1}{1+0.135} \approx 0.880$$

Interpretation: 88% probability of class $+1$; since $0.88 > 0.5$, predict $y = 1$.

Q2 (Medium): Prove that the sigmoid satisfies $1 - g(z) = g(-z)$.
Proof: $g(-z) = \frac{1}{1+e^z}$. $1 - g(z) = 1 - \frac{1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}} = \frac{1}{e^z+1} = \frac{1}{1+e^z} = g(-z)$. $\square$

This symmetry means the sigmoid is symmetric around the point $(0, 0.5)$.

Topic: Cross-Entropy Loss

Q3 (Easy): A model predicts $h = 0.9$ for a positive example ($y = 1$). What is the cross-entropy loss?

$$\text{Loss} = -1 \cdot \log(0.9) - 0 \cdot \log(0.1) = -\log(0.9) \approx 0.105$$

Small loss — a confident and correct prediction.

Q4 (Medium): A model predicts $h = 0.2$ for a positive example ($y = 1$). Compute the loss and explain why it is so large.

$$\text{Loss} = -\log(0.2) \approx 1.61$$

The model is 80% confident *against* the true class. Cross-entropy severely penalises this because the log function approaches $-\infty$ as its argument approaches 0.

Q5 (Exam-level): Derive the gradient of the binary cross-entropy loss with respect to $\mathbf{w}$ for a single training example. State every step clearly using the chain rule, and show why the $g(1-g)$ terms cancel.
See the Derivation box in Section 3.4 for the complete proof.

Topic: Logistic Regression GD

Q6 (Medium): Using $\mathbf{w} = [1,\ -1]$, $\boldsymbol{\phi}(\mathbf{x}) = [1,\ 2]$, $y = 0$:
(a) Compute $h(\mathbf{x})$. (b) Compute the prediction error. (c) Write the gradient contribution from this example. (d) Perform one SGD step with $\alpha = 0.1$.
Solution:
(a) Score $= 1(1) + (-1)(2) = -1$; $h = g(-1) = 1/(1+e) \approx 0.269$.
(b) Error $= h - y = 0.269 - 0 = 0.269$.
(c) Gradient $= 0.269 \times [1,\ 2] = [0.269,\ 0.538]$.
(d) $\mathbf{w} \leftarrow [1, -1] - 0.1[0.269, 0.538] = [0.973,\ -1.054]$.

Q7 (Exam-level): Compare logistic regression and the hinge-loss linear classifier on the following four dimensions: (i) output type, (ii) loss function, (iii) gradient sparsity, (iv) probabilistic interpretation. State which you would prefer for a medical diagnosis application where the confidence of the prediction matters and justify your answer.
Answer:

(i) Logistic: probability in $(0, 1)$; hinge: hard label ± 1.
(ii) Logistic: smooth cross-entropy; hinge: piecewise linear.
(iii) Hinge gradients are *sparse* (zero when margin ≥ 1, useful for SVMs); logistic gradients are always non-zero (slow but stable).
(iv) Only logistic regression has a principled probabilistic interpretation backed by MLE.

Prefer logistic regression for medical diagnosis: A doctor needs to know not just "positive/negative" but *how likely* the diagnosis is (e.g. 92% vs. 51%) to make informed decisions about further testing. Hinge loss gives no such probability.

7 Key Takeaways — Week 3

Quick Summary

- **Logistic hypothesis:** $h(\mathbf{x}) = g(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}))$ where $g(z) = 1/(1+e^{-z})$.
- **Sigmoid range:** $(0, 1)$; output = probability of positive class; $g(0) = 0.5$.
- **Sigmoid derivative:** $g'(z) = g(z)(1 - g(z))$ — elegant and compact; enables clean gradient derivation.
- **Binary Cross-Entropy / Log Loss:** $-y \log h - (1-y)\log(1-h)$. Convex; theoretically motivated by MLE. Heavily penalises confident wrong predictions.
- **Training Loss:** $-\frac{1}{N}\sum[y^{(i)} \log h^{(i)} + (1-y^{(i)})\log(1-h^{(i)})]$.
- **Gradient:** $\frac{1}{N}\sum(h(\mathbf{x}^{(i)}) - y^{(i)})\mathbf{x}^{(i)}$ — same form as linear regression.
- **GD Update:** $\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N}\sum(h^{(i)} - y^{(i)})\mathbf{x}^{(i)}$.
- **Logistic vs. Hinge:** Cross-entropy gives probabilities and smooth gradients; hinge gives hard labels and sparse sparse updates.
- All three models (linear regression, hinge classifier, logistic regression) share the same gradient *form*: (prediction − truth) × features. Only the definition of "prediction" changes.